Volatility Forecasting with Transformers for Deep RL Intraday Trading in ES Futures

By Guillermo Alfonso Medrano Sanchez \ 高文昊
Machine Learning and Econometrics
Instructor: Dr. 冼芻蕘
National Tsing Hua University
December 2025

Code available at https://github.com/GGHOSTRADER/ML_econ

Abstract

Classical Machine learning for forecasting returns of assets have not been very successful mostly due to its stochastic nature that is non monotonic and can be not linear. While manual traders try a more intuitional approach which can outperform models at times when there is not enough data to forecast, they suffer from bias and lack empirical statistics.

Mini ES future contracts(SP500 future contracts) were used as the asset for the research and the granularity chosen of the bars (the information aggregation method from broker which is Tradestation) was 1 minute bars that have OHLCV data and VWAP.

Tim Bollerslev proposed that volatility exhibits clustering/persistence and is often modeled via ARCH/GARCH, which then lead me to use modern architectures that can read multi step context like Transformers and LightGBM (as comparison of performance) to forecast volatility and then "replacing" the manual trader intuition with a model that can act as a trader but sticking to objective measures, therefore a deep reinforcement learning architecture which acts as the manual trader. The results of using Sequential models and Agentic RL based models greatly outperformed the PnL of buy-and-hold while having a very good risk profile, therefore excelling promising results for trading applications

Introduction

In intraday operations, Volatility is a very important variable, just or more important than direction. This is due that even if you get correctly forecasted the direction, under forecasting the volatility means carrying more risk at the wrong moment which can lead to a overleveraged and oversized position that can wipe out our entire account and funds. Over forecasting volatility means we may be trading with less risk than we should be doing, trumpeting our returns in the long term, if we don't increase size when we should, our returns will be very low.

In other words, if we can have a proper forecast of volatility, we can manage our risk, protect our capital and have better returns.

Markets are full of noise which are fall alerts either given by the illusion of patterns or by actual algorithms designed to fool retail investors. Markets also have very fast data at a microstructure level which retail investors cant access and the nonstationary of it makes it hard to forecast in a naive way.

All this noise and speed make retailer traders who trade manually also have a very hard time being able to constantly perform well. Non only machines are faster, have no emotions and don't get tired but also are able to take decision more objective way, but machines do struggle in capturing regime changes unless the designer of the model has high domain knowledge which manual traders do have. Sometimes we get great models that can perform really well when a regime is stable but their performance quickly erodes when conditions change or we get a great trader which intuition that can change its trading quickly but gets tired and falls into fake patterns since it is not necessarily using objective methods all the time.

Retail investors usually go unsophisticated methods to forecast volatility and risk, one of the most common practices is calculating a ATR and then using the average of it, Higher the ATR (compared to the levels in this asset) will make the retailer trader have bigger stop loss and profit taking, also put less or more size. This approach is very slow and late since is based on a moving average which smooths the data, but when it comes to volatility, smoothing data paired with discrete data aggregations like 1 minute bars, means retail investors get signals of changes in volatility several bars/minutes after it happened, which result in not being able to react in time.

Manual traders which are at the front lines, can sense the change in market regimes and can identify new opportunities for building strategies but this traders have a constrain of resources per day (time, energy, light of sight), their approach is not scalable, since trading a new trader is very hard and the already trained one cant increase the hours a day they work. Therefore a method or model that can replicate this process they do but also can do parallelism and do it faster would squeeze more out of their approach.

Data

The asset being used for this study is E-MINI SP500 which is the future contract for the weight index know as standard & poor 500 . The institution responsible of this futures is CME (Chicago mercantile exchange), they are the sole issuers of the futures and centralize the volume, meaning that all the volume traded has to shown in CME transactions. The broker which the data was extracted from is Tradestation which works with CME.

The data is aggregated into 1 minute intervals , called OHLCV bars, this bars summarize what happened in 1 minute of trading. The data from Tradestation also included a pre calculated VWAP.

$$\mathrm{VWAP}_t = \frac{\sum_{i=1}^{t}\left(\frac{H_i+L_i+C_i}{3}\right)V_i}{\sum_{i=1}^{t} V_i}$$

V = **Volume** (shares/contracts traded)
H = **High** price of the bar
L = **Low** price of the bar
C = **Close** price of the bar

Time span
The data was recollected is from 08-23-2023 to 12-08-2025 with more than 800,000 data steps/rows.

Basic preprocessing:
Raw data are loaded, timestamps are standardized into a sorted datetime index, and essential base variables (volume, log returns, and ATR) are constructed when missing; no rows are dropped prior to feature engineering.

1. **Volume**
   $V = UpV + DownV$
   V = Volume
2. **Log Returns**
   $r_t = \log\left(\frac{C_t}{C_{t-1}}\right)$
   C = Close
3. **ATR**
   $\mathrm{TR}_t = \max(H_t - L_t,\ |H_t - C_{t-1}|,\ |L_t - C_{t-1}|)$
   $\mathrm{ATR}_t^{(n)} = \frac{1}{n}\sum_{i=0}^{n-1}\mathrm{TR}_{t-i}$
   H= High
   L = Low
   C = Close

Methodology

Feature engineering is needed to encode the behavior of past volatility and volume for the models to learn from it. Volatility is closely related to volume since a more liquid order book will tend to be more volatile but also high influx of market orders volume can also cause volatility. Therefore most of the features created are describing the context of the market of volatility, volume or time. Time is also important since there are particular moment of the day that will

have more volume (the opening hour and closing hours of USA trading hours). Some of the features are created to point out it is that moment of the day.

Features engineered

1. `parkinson_vol_5`
   Objective: Capture *very short-term intrabar volatility* using high–low information, which is more efficient than close-to-close returns at high frequency.
   $\text{parkinson\_vol\_5} = \sigma_{P,t}^{(5)}$
2. `parkinson_vol_15`
   Objective: Measure *short-horizon volatility* aligned with the prediction horizon, smoothing microstructure noise.
   $\text{parkinson\_vol\_5} = \sigma_{P,t}^{(15)}$
3. `parkinson_vol_30`
   Objective: Provide a *medium-term volatility context* to detect volatility clustering and regime persistence.
   $\text{parkinson\_vol\_5} = \sigma_{P,t}^{(30)}$

Formula
$\sigma_{P,t}^{(w)} = \sqrt{\frac{1}{4\log 2} \cdot \frac{1}{w} \sum_{i=0}^{w-1} \left[\log\left(\frac{H_{t-i}}{L_{t-i}}\right)\right]^2}$
H = High
L = Low
w = Rolling window size

4. `ofi_5` **(Order Flow Imbalance)**
   Objective: Capture *immediate buying vs selling pressure* driven by aggressive trades.
   $\text{ofi\_5} = \text{OFI}_t^{(5)}$
5. `ofi_15`
   Objective: Measure *sustained directional pressure* over a short window rather than single-bar noise.
   $\text{ofi\_15} = \text{OFI}_t^{(15)}$
6. `ofi_30`
   Objective: Detect *persistent order-flow dominance*, often preceding volatility expansion.
   $\text{ofi\_15} = \text{OFI}_t^{(30)}$

Formula
$\text{OFI}_t^{(w)} = \sum_{i=0}^{w-1} (\text{Up}_{t-i} - \text{Down}_{t-i})$
Up = Buying Market Orders
Down = Selling Market Order

7. `volume_percentile`
   Objective: Express current volume in *relative terms* (unusual vs normal), making volume regime-aware instead of scale-dependent.

Formula

$$\text{volume\_percentile}_t = \frac{\text{rank}\left(V_t | \mathcal{V}_t^{(60)}\right) - 1}{|\mathcal{V}_t^{(60)}| - 1}$$

This maps the volume to [0,1]

8. `volume_momentum`
   Objective: Capture *changes in trading activity intensity*, signaling information arrival or liquidity shocks.

Formula

$$\text{volume\_momentum}_t = \frac{V_t - V_{t-5}}{V_{t-5}}$$

V= Volume

9. `amihud_illiquidity`
   Objective: Measure *price impact per unit of volume*, identifying low-liquidity conditions where volatility is amplified.

Formula

$$\text{amihud\_illiquidity}_t = \frac{|r_t|}{V_t}$$

V = Volume

r = asset return (log return in this case)

10. `vwap_distance`
    Objective: Quantify *price dislocation from fair value* (VWAP), normalized by ATR to remain volatility-consistent.

Formula

$$\text{vwap\_distance}_t = \frac{C_t - \text{VWAP}_t}{\text{ATR}_t}$$

C = Close

11. `minutes_since_open`
    Objective: Encode *intraday seasonality*, since volatility and liquidity follow strong time-of-day patterns.

Formula

$$\text{minutes\_since\_open}_t = \max\left(0, \frac{t - t_{\text{open}}}{60}\right)$$

t = time / minutes

13. `is_first_last_30min`

    Objective: Flag *structural market regimes* (open/close) where volatility, spreads, and order flow behave fundamentally differently.

Formula

$$\text{is\_first\_last\_30min}_t = \begin{cases} 1, & 0 < m_t \leq 30 \text{ or } 360 < m_t \leq 390 \\ 0, & \text{otherwise} \end{cases}$$

m = minutes since open

Target Variable

Realized volatility

Realized volatility at horizon 15 measures the **total price variability accumulated over the next 15 bars** (15 minutes), computed from squared returns and then square-rooted.

Is a forward looking metric, so uses the subsequent 15 steps(bars/minutes) to calculate it. I am encoding the volatility behavior of the next 15 minutes of trading.

Formula

$$\text{RV}_t^{(15)} = \sqrt{\sum_{i=1}^{15} r_{t+i}^2}$$

r = Return

Transformations

Numbers per engineered feature

Feature	Skewness	Kurtosis
parkinson_vol_5	2.1629	7.0466
parkinson_vol_15	2.0017	5.7764
parkinson_vol_30	1.9152	5.3728
ofi_5	2.5565	79.4336
ofi_15	-0.2772	22.1160
ofi_30	-0.3307	11.1740
volume_percentile	0.0238	-1.2934
volume_momentum	3.4933	16.7653
amihud_illiquidity	11.2938	301.1575
vwap_distance	3.5041	11.7656

Feature	Skewness	Kurtosis
minutes_since_open	0.6707	-1.1141
is_first_last_30min	4.5796	18.9755

Kurtosis in Pandas

- 0 ⇒ normal-like tails
- Bigger than 0 ⇒ heavier tails than normal
- Smaller than 0 ⇒ lighter tails than normal

Skewness in Pandas

- Skewness = 0 ⇒ *perfect symmetry*
- Positive skew ⇒ long right tail
- Negative skew ⇒ long left tail

1. Log transform (`log1p`)

Purpose:
Reduce right-skew / heavy tails, stabilize scale, dampen extreme spikes while preserving ordering. one of the properties of log transformations are that they only accept positive numbers, so this transformation is only applied to right skewed features.

$x \mapsto \log(1 + \max(x, 0))$

Applied to:

- `parkinson_vol_5`
- `parkinson_vol_15`
- `parkinson_vol_30`
- `amihud_illiquidity

Why these:
All are strictly non-negative and highly skewed with occasional extreme values.

2. Quantile clipping
   Purpose:
   Prevent extreme outliers from dominating scaling and model gradients. Basically it caps what an extreme value is therefore once is outside the percentile, does not if the number is bigger by 10 or by 10000, it will be the same extreme value. Keeps the distribution shape but very dipropionate outliers will be capped.

$x \mapsto \min(\max(x, q_{0.005}), q_{0.995})$

Applied to:

- `vwap_distance`
- `volume_momentum`

Normalization

1. Time normalization (train-max scaling)

Purpose:
Convert clock time into a relative intraday position without assuming a fixed session length.

$m_t \mapsto \frac{m_t}{\max_{\text{train}}(m)}$
Applied to:

- `minutes_since_open`

2. Robust scaling (median / IQR)

Purpose:
Put continuous features on comparable scales while remaining insensitive to outliers.

$x \mapsto \frac{x - \text{median}(x)}{\text{IQR}(x)}$
IQR = Inter Quantile Range

Robust Scaling vs Standard Scaling.

Standard Scaling

It relies con normal distribution, therefore using it in heavily tailed distributions will be incorrect. We have several heavy tailed distributions.
$x = \frac{x-\mu}{\sigma}$
$\mu = mean$
$\sigma = variance$

Robust scaling relies on median and not Mean, which makes it better option to deal with distribution with heavy outliers.

In a few words, several engineered features exhibit substantial skewness and excess kurtosis, reflecting heavy-tailed market behavior. Right-skewed variables are stabilized using log transformations or quantile clipping, while robust normalization is applied to continuous features to ensure comparable scale without altering tail structure. Binary regime indicators are left untransformed.

why is normalization needed?
I normalize features so that **no single feature dominates the model purely because of its scale**, rather than because it carries more information.

Without normalization, the transformer will still run — but it will learn **worse, slower, and less stably**. Transformers rely on gradient-based optimization which is very sensitive to scale.

The goal is to forecast the realized Volatility of the next 15 minutes / bars, so for this an encoder only transformer was used.

Transformer architecture

This transformer only has an encoder layer, this is **not** a full “NLP-style” transformer. It’s a **time-series encoder-only transformer** that:

- takes a **sliding window of engineered features**
- models **temporal dependencies across time**
- outputs **one scalar volatility forecast** for the **last time step only**

No decoder, no autoregression, no attention over outputs. In other words, we feed it the 12 features we engineered, the target variable we created and it learns to predict the 15 min realized volatility. The output is a continuous number that represents the predicted realized volatility that will happen through the next 15 minutes / bars.

Summary of the Transformer hyperparameters and specs

Component	Value
Sequence length	**120 bars**
Features	**12**
Model dimension (`d_model`)	**128**
Attention heads	**4**
Encoder layers	**3**
FFN hidden size	**256**
Dropout	**0.1**
Output activation	**Softplus**
Optimizer	**Adam**
Learning rate	**1e-4**

Component	Value
Weight decay	**1e-5**
Batch size	**256**
Epochs	**6**
Loss	**QLIKE (σ-space)**

As we can see from the table, the chosen Loss Function for the Transformer was QLIKE over the commonly used MSE. Model was first trained using MSE I will now proceed to explain why QLIKE is a better loss function than MSE.

1. MSE
   MSE = $\frac{1}{T}\sum_{t=1}^{T}(\hat{y}_t - y_t)^2$

But of the properties of the error being squared, both
$(\hat{y} - y)^2 = (y - \hat{y})^2$

give the same result, which in forecasting volatility is not the correct way to approach the forecast. Because under forecasting volatility may lead to overleverage/sized position in the wrong moment and it will expose my capital which can potentially cause a devastating loss while over forecasting will mostly mean under sizing at the wrong moment which will translate to losing an opportunity but in this industry, capital conservation is by far more important than getting all opportunities.

This is why I introduce the usage of QLIKE for my model, a loss function usually used in quantifying risk.

2. QLIKE
   QLIKE = $Log(\hat{y}) + \frac{y}{\hat{y}}$

QLIKE only accepts positive numbers due to the $Log(\hat{y})$ on the equation which is no problem since volatility cannot be negative, its error can and from the start, this prevents the forecast of being incorrectly forecasted as negative. Second, the part of the equation $\frac{y}{\hat{y}}$ also plays a very important role.

Example
$y = 0.04$

Under forecasted : $\hat{y} = 0.001$
Over forecasted : $\hat{y} = 0.079$

Distance of both : $d = 0.039$

MSE:

Under
$(0.04 - 0.001) = 0.039$
$(0.039)^2 = 0.001521$

Over
$(0.04 - 0.079) = -0.039$
$(-0.039)^2 = 0.001521$

Both errors type of errors excel the same number.

QLIKE:

-Under:
$log(0.001) \approx -6.9078$
$\frac{0.04}{0.001} = 40$
$-6.9078 - 40 = 33.0922$

-Over
$log(0.079) \approx -2.538$
$\frac{0.04}{0.079} \approx 0.506$
$-2.538 + 0.506 = -2.032$

Clearly in the results speak for their self.

MSE both sides had an error of **0.001521** while QLIKE had dramatically different errors, Under-forecast: **33.09** and Over-forecast: **−2.03**, both have same distance but the side of it makes the error manageable or a huge number, in other words punishment is different depending on the error side.

This are the summarized steps of the transformer

1. **Input**
   - Take last **120 bars × 12 normalized features**
2. **Project**
   - Linear map: `12 → 128` (feature embedding)
3. **Add time info**
   - Add **sinusoidal positional encoding**
4. **Encode sequence**
   - **3 Transformer encoder layers**
     - 4-head self-attention
     - FFN: `128 → 256 → 128`
     - LayerNorm + residuals
5. **Collapse time**
   - Keep **last time step only**

6. **Predict**
   - MLP: `128 → 64 → 1`
   - `Softplus` → positive volatility
7. **Train**
   - Minimize **QLIKE** on scaled target
8. **Evaluate**
   - Rescale outputs → compute **true-scale QLIKE**

> *The model is based on the Transformer encoder architecture introduced by Vaswani et al. (2017). Detailed explanations of self-attention and positional encoding can be found in the original work.*

> *An implementation-level explanation is provided by the Annotated Transformer (Harvard NLP).*

LightGBM

In parallel, a lightGBM model was also trained which is a boosting tree based model which handles structured data very well, has a long track of being used in financial predictions and can be much easier for explanations.

Summary of the Transformer hyperparameters and specs

Category	Hyperparameter	Value	What it Controls
Objective	`objective`	`regression`	L2 regression loss used for training (predicts volatility σ)
Evaluation	`metric`	`None`	Disables built-in metrics (custom QLIKE used instead)
Evaluation	`feval`	`qlike_metric`	Custom QLIKE metric used for validation and early stopping
Optimization	`learning_rate`	`0.05`	Step size for each boosting iteration
Model Capacity	`num_leaves`	`64`	Maximum number of leaves per tree (controls tree complexity)
Regularization	`min_data_in_leaf`	`100`	Minimum samples per leaf (prevents overfitting)
Feature Sampling	`feature_fraction`	`0.8`	Fraction of features randomly selected per tree

Category	Hyperparameter	Value	What it Controls
Row Sampling	`bagging_fraction`	`0.8`	Fraction of samples randomly selected per tree
Row Sampling	`bagging_freq`	`1`	Apply bagging every boosting iteration
Boosting	`num_boost_round`	`200`	Maximum number of boosting iterations (trees)
Early Stopping	`stopping_rounds`	`100`	Stop if validation QLIKE does not improve for 100 rounds
Logging	`log_evaluation`	`100`	Print evaluation metrics every 100 iterations
Logging	`verbosity`	`-1`	Suppress LightGBM internal output

Summarized steps of the LightGBM

Input
- 12 normalized features

1. **Initialize model**
   - Start with a constant prediction (mean of the target).
2. **Iterative boosting loop**
   - Repeat for each boosting round:
3. **Compute gradients**
   - Calculate first- and second-order gradients of the loss for all samples.
4. **Grow a decision tree**
   - Use **leaf-wise growth** (not level-wise).
     - Choose splits that maximize **loss reduction** using gradients and Hessians.
     - Apply regularization constraints (min samples per leaf, max leaves).
5. **Update ensemble**
   - Add the new tree to the model.
   - Scale its contribution by the learning rate.
6. **Evaluate model**
   - Compute evaluation metrics on training and validation data.
7. **Early stopping (optional)**
   - Stop boosting if validation performance stops improving.
8. **Final prediction**

We employ LightGBM, a gradient boosting decision tree framework that builds an additive ensemble of decision trees using leaf-wise growth for efficient loss minimization (Ke et al., 2017).

Backtesting Engine

The backtesting basically is a bar to bar simulation engine of historical data that operates with time indexed data, it models entries of trades, exists, position, PnL and can operate with a determined rule or strategy it is given.

It checks at each time step the following:

- queries a user-defined **policy function** for the desired target position,
- executes any required position change **at the OPEN price of the current bar**,
- applies **explicit transaction costs** (fixed commissions and volume-based slippage),
- computes realized PnL from **OPEN-to-OPEN price changes**, a- updates portfolio cash and equity accordingly.

It keeps track of all events and bar level results, then it aggregates them for the final results. The engine produces a equity curve through the trades, summary table of each trade and performance derived from the equity curve.

Outputs of the backtesting engine

Output Category	Name	Description
Time Series	`equity_curve`	Portfolio cash/equity over time after each bar, net of transaction costs
Time Series	`positions`	Number of contracts held at each bar
Time Series	`cash_series`	Cash balance at each bar after PnL and costs
Time Series	`pnl_series`	Bar-level realized PnL (OPEN→OPEN), net of transaction costs
Trade Log	`trades`	Raw execution events recording position changes, prices, timestamps, and costs
Trade Summary	`trades_df`	Round-trip trade table with entry/exit times, prices, direction, size, and net PnL
Summary Stats	`summary`	Dictionary of aggregate performance metrics computed from equity path

Output Category	Name	Description
Plot	`equity_curve_last3m.png`	Trade-based cumulative PnL plot (last 3 months)
File	`trades_summary_last3m.csv`	CSV file containing round-trip trade summaries

This engine is crucial for running the simulations for the next step in the pipeline which is deep reinforcement learning.

Deep Reinforcement Learning

Reinforcement learning (RL) is formulated as a Markov Decision Process, an agent learns a policy to maximize expected cumulative reward through respective interaction with an environment. RL works really well on deterministic games that are not stochastic by nature, that's why we needed to use a more complex version or RL which uses NN and learns not only to play but from interactions with the game.

The used approach builds on modern actor–critic methods with entropy regularization. SAC is an off-policy deep reinforcement learning method that augments the expected return objective with an entropy term, encouraging exploration and improving training stability. This tries to mimic the behavior of a manual trader who will try new strategies and not just learn the best actual one, the market is not like playing a controlled game like chess where rules and pieces are always the same, regime changes, laws changes, so it is important that the agent tries to learn new things and not just zero in into the best actual one.

In this architecture of SAC, the actor selects actions according to a stochastic policy, while the critic evaluates these actions via learned Q-functions; the critic’s value estimates guide policy updates, and entropy regularization ensures stable exploration. The model is given $100,000 and it should trade E-MINI ES contracts.

Summarized steps

input:

- The deep reinforcement learning agent observes the 12 engineered features, model-based volatility forecast (what we forecasted before), and the current position[-1,0,1] observed by the policy network.

1. **Environment defines the RL problem (one day = one episode)**

- When an episode starts, it randomly picks a **trading day**, takes that day's 1-minute rows, and sets:
  - `position = 0`, `cash = 0`, `idx` at first bar.
- **State at time t** = `[13 features at t] + [current position]` → 14 numbers.

2. **Actor chooses an action (policy)**
   - Actor takes the 14-dim state and outputs 3 probabilities for `{flat, long, short}`.
   - Training uses **stochastic sampling**, so it keeps exploring.
3. **Action becomes a target position**
   - Action is mapped to desired position in `{ −1, 0, +1 }` (and clipped by `max_position=1`).
4. **Costs are charged immediately at the current OPEN**
   - Compute **position_change = desired_pos - current_pos**
   - Apply:
     - fixed commission + a crude **sqrt impact** slippage model using current bar volume.
5. **PnL is computed one bar ahead using the NEW position**
6. **Reward is emitted and stored**
   - Reward Stored
   - Transition `(state, action, reward, next_state, done)` goes into replay buffer.
7. **SAC update happens every step (after buffer is big enough)**
   - **Critic update:** learns Q-values so that `Q(s,a)` matches a bootstrapped target:
     - target uses the **target critic** and the actor's full distribution over next actions.
     - entropy term `− alpha * log pi(a|s)` is included → encourages exploration.
   - **Actor update:** shifts the policy to put more probability on actions with high Q,
   - **Target critic soft update:** slowly tracks critic with `tau=0.005`.
8. **End of day**
   - When the environment reaches the last usable bar in the day, episode ends.
   - Log:
     - `ep_reward` (sum of scaled rewards) and `ep_pnl` (sum of pnl-cost in dollars).
9. **Outputs**

- learning curve + daily PnL proxy

We employ a discrete Soft Actor-Critic (SAC) agent (Haarnoja et al., 2018; Christodoulou, 2019), an entropy-regularized actor–critic method within the standard reinforcement learning framework (Sutton and Barto, 2018).

Summary of the Transformer hyperparameters and specs

Component	Parameter	Value in code
Data	Lookback window	last **3 calendar months**
	Joined columns	`Open`, `Volume` + **13 features**
State	Feature dims	**13**
	Position in state	**+1 dim**
	`state_dim`	**14**
Actions	Discrete action set	**3 actions**
	`action_dim`	**3**
Episode	Episode definition	**one trading day**
	Terminal condition	last usable bar is `idxs[-2]`
Position constraints	`max_position`	**1**
Pricing / PnL model	Price used	**Open-to-Open**
	Contract multiplier	**50.0**
Transaction costs	Commission	**$2.50 per contract change**
	Slippage model	sqrt impact
	Participation rate	`abs(position_change)/daily_volume`
Reward	Reward definition	`(pnl - cost) / reward_scale`
	`reward_scale`	**100.0**
Replay buffer	Capacity	**100,000**
	Sampling	Uniform random
SAC (discrete)	Algorithm	**Discrete SAC-like**
Actor network	Hidden size	**256**
	Layers	Linear → LN → ReLU → Linear → LN → ReLU → Linear
	Output	`logits`, `softmax(logits)`
	Action selection	stochastic
Critic network	Type	**Twin Q networks**

Component	Parameter	Value in code
	Input	`[state, one_hot(action)]`
	Hidden size	**256**
	Layers per Q	Linear → ReLU → Linear → ReLU → Linear(1)
Target network	Target critic	yes
	Soft update	**tau = 0.005**
Optimization	Optimizer	Adam
	Learning rate	**3e-4**
Discount / entropy	`gamma`	**0.99**
	`alpha`	**0.01**
Batching	`batch_size`	**256**
Training loop	Episodes	**200**
	Updates per step	**1**
	Warm start	none
Logging / outputs	Log CSV	`rl_training_log.csv`
	Saved models	`actor_sac_daily.pt`, `critic_sac_daily.pt`

A model was trained using Transformer volatility forecasts and a second model was trained using LightGBM volatility forecasts, beside the volatility forecast, they shared all other features and conditions.

Experiments & Results

Transformer Training

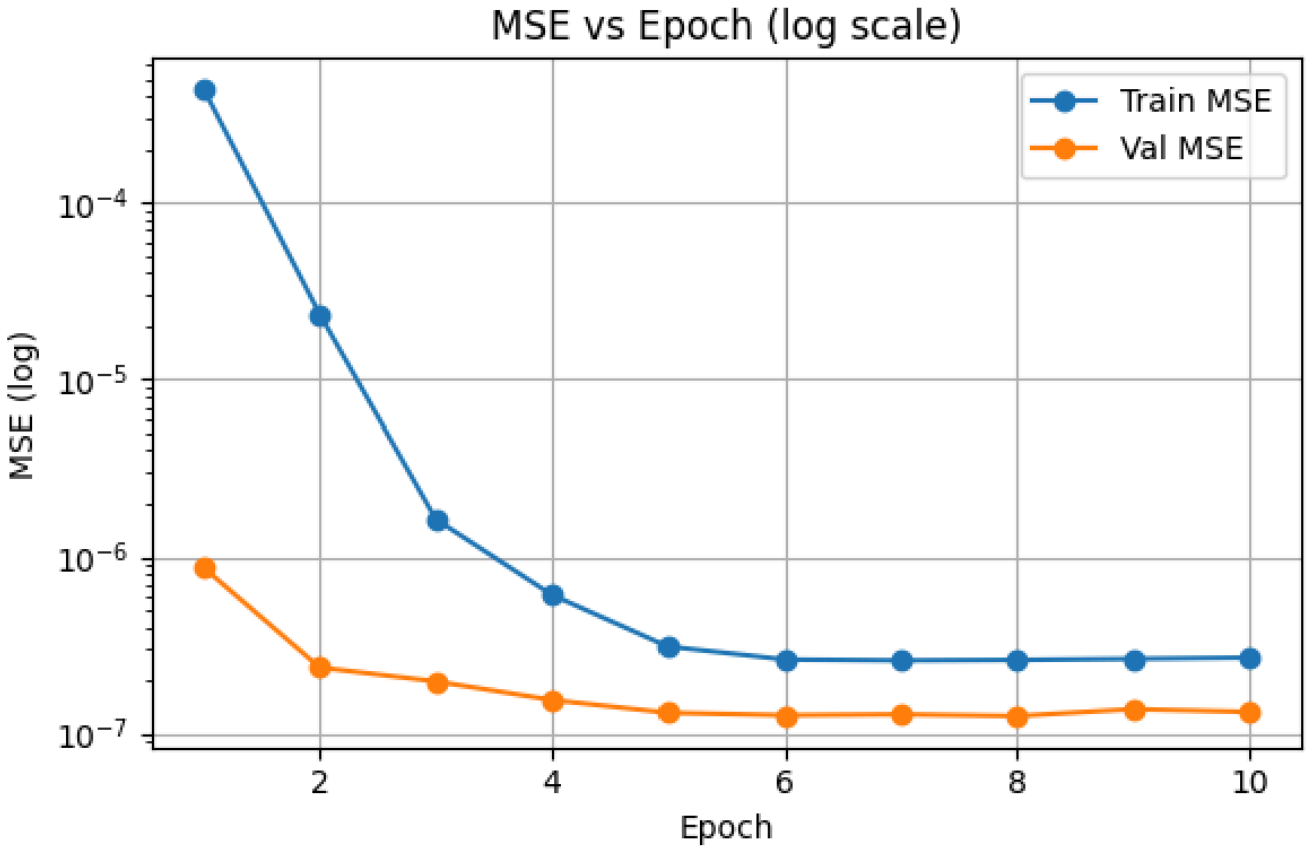

Figure 1

Figure 1 shows training and validation curve of transformer training using MSE as loss function

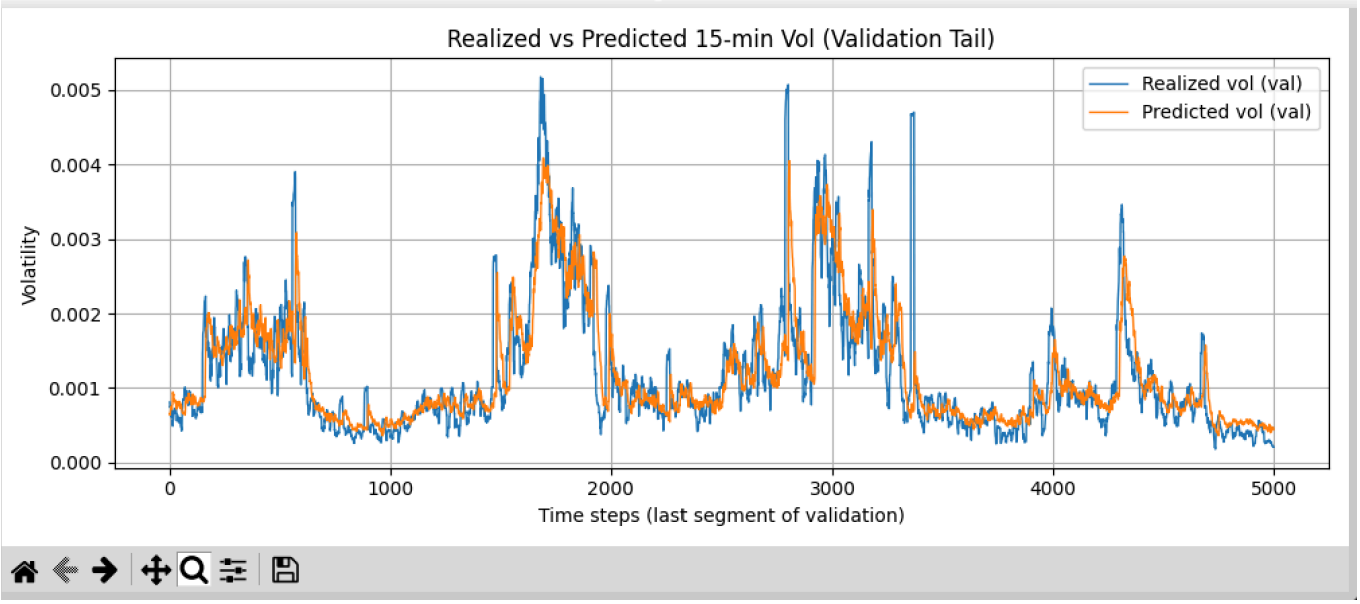

Figure 2

Figure 2 shows transformer prediction compared to realized volume while using MSE as loss function.

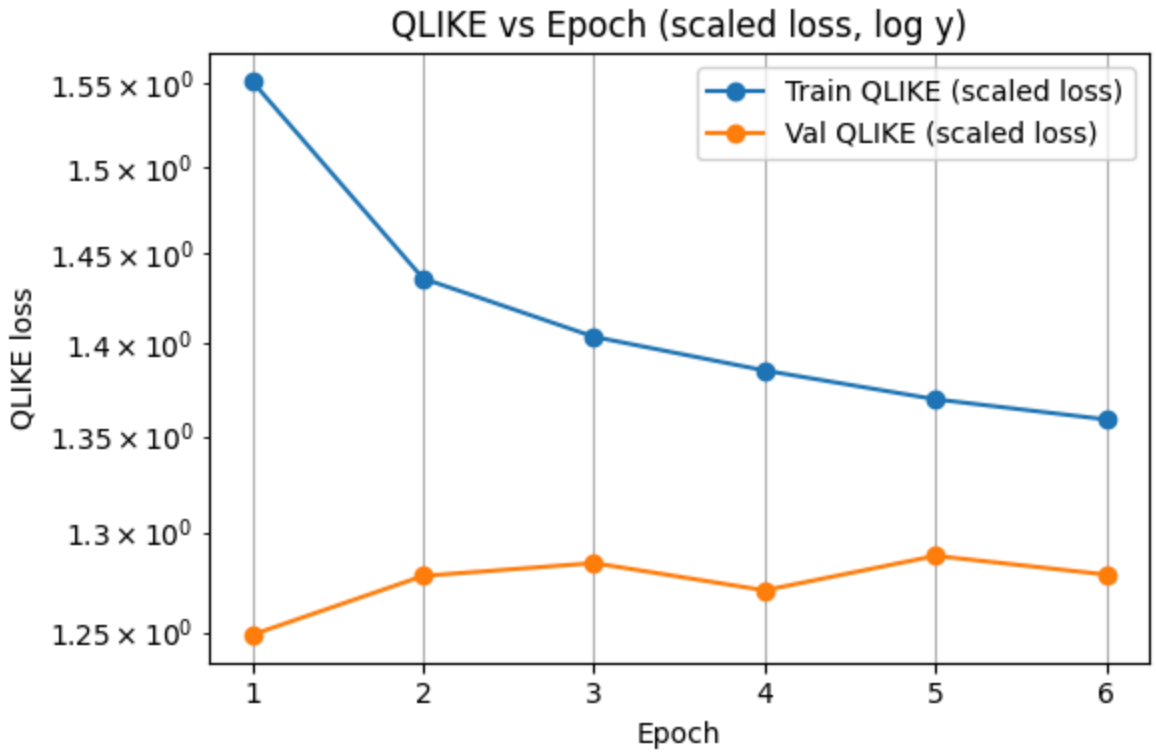

Figure 3

Figure 3 shows Transformer training and validation curves using QLIKE as loss function

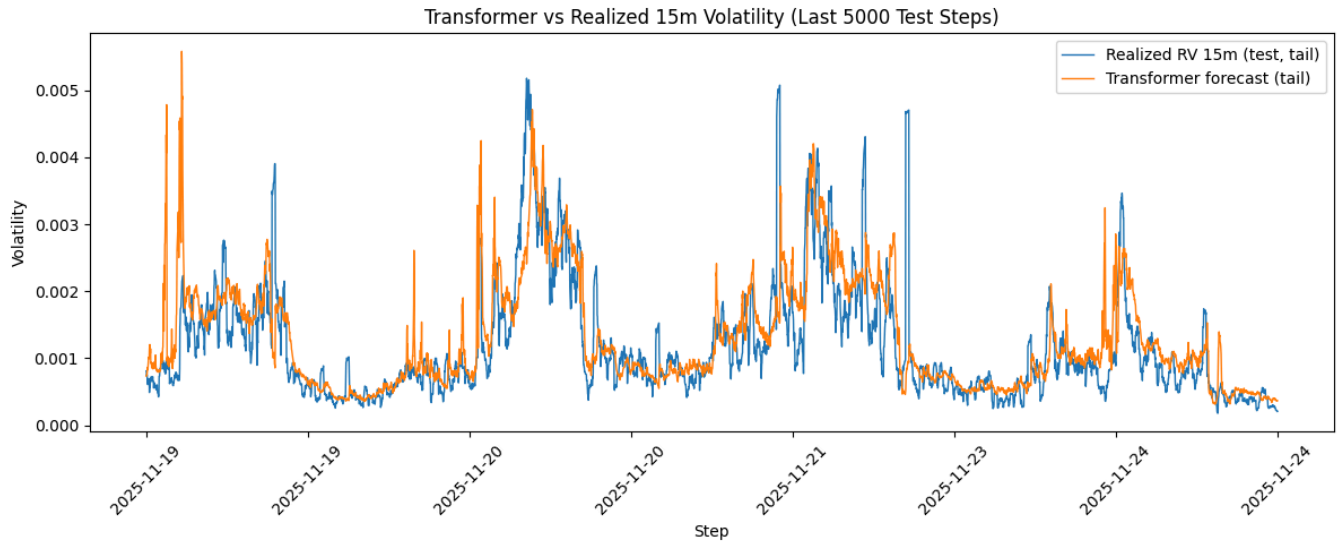

Figure 4

Figure 4 shows Transformer prediction compared to realized using QLIKE

LightGbm Training

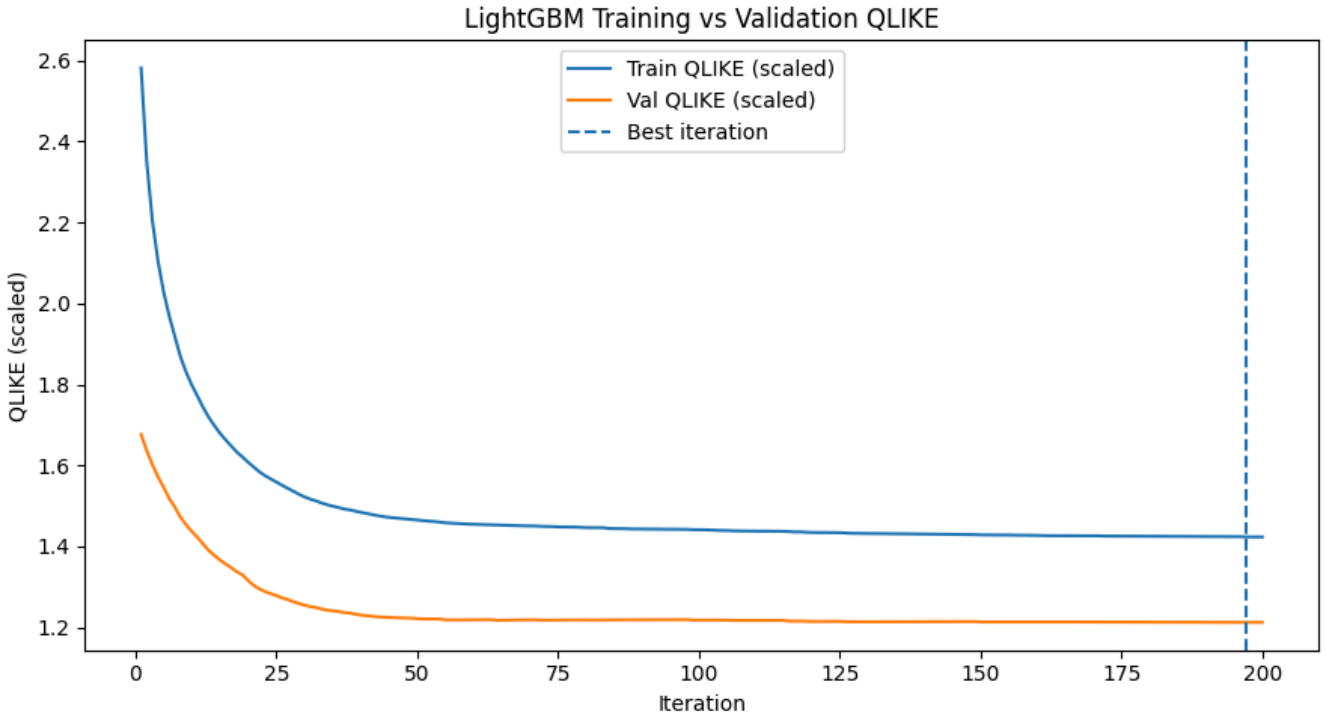

Figure 5

Figure 5 shows Training and Validation curve of LightGBM training using QLIKE

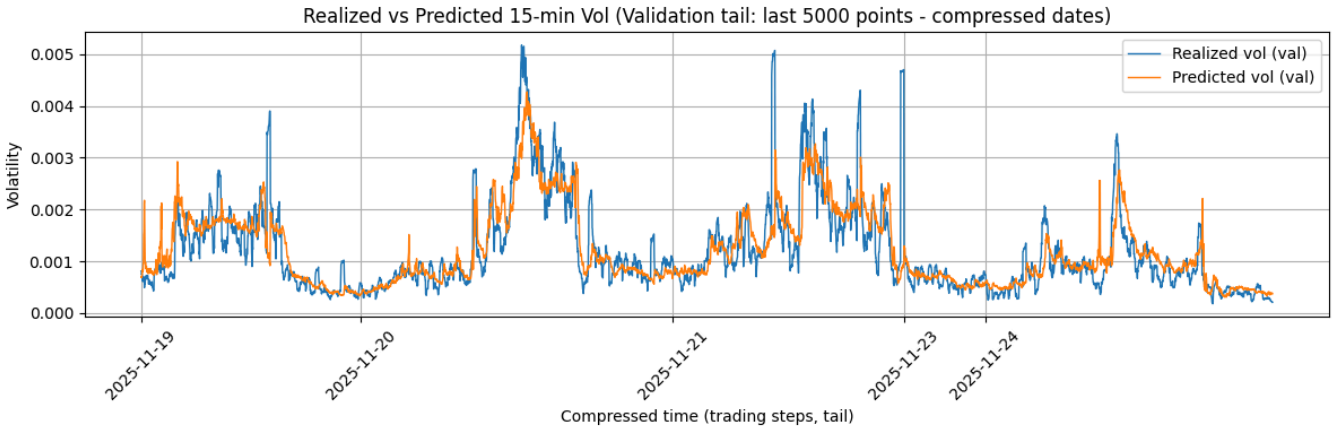

Figure 6

Figure 6 shows LightGBM prediction compared to realized using QLIKE

Quantitative measures comparing LightGBM QLIKE and Transformer QLIKE

Metric	LightGBM	Transformer	Winner
Corr (lag 0)	0.7807	0.7418	LightGBM (slightly)
Best Corr	0.8836 @ 15 bars	0.7951 @ 12 bars	LightGBM
Lag to best alignment	15 bars	12 bars	Transformer
Spike recall	0.6415	0.6555	Transformer

Metric	LightGBM	Transformer	Winner
Spike precision	0.6459	0.5615	LightGBM

Figure 7

Deep Reinforcement Learning

DRL + Transformer

Deep Reinforcement Learning using initial 12 features and Transformer forecasted realized 15 minutes

Metrics

Metric	Value	Commentary
Round-trip trades	**1,508**	higher turnover
Long trades	922	~61% long
Short trades	586	~39% short
Win % (net)	**69.8%**	Good winning %
Gross PnL	**$96,450**	Strong raw signal
Total Costs	**$18,115**	High do to a lot of trades
Net PnL	**$78,335**	Good PnL
Profit Factor (net)	**4.01×**	Excellent
Avg Win (net)	$99.14	Good compared to losses
Avg Loss (net)	−$57.27	Good
Max Drawdown	**$828**	Shockingly low
Median Hold Time	1 bar	Scalping regime
Avg Hold Time	2.76 bars	Slightly longer
Max Hold Time	49 bars	~49 minutes
Avg trades/day	**116**	Heavy trading

Figure 8

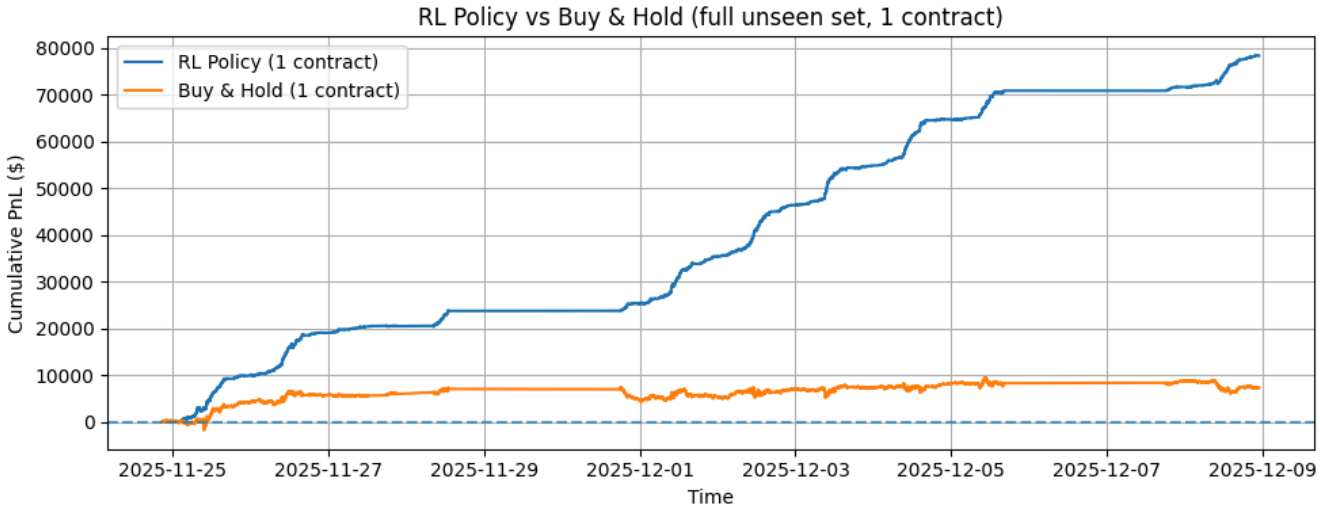

Figure 9

Figure 9 shows Equity curve of Deep Reinforcement Learning using initial 12 features and Transformer forecasted realized 15 minutes and Buy and hold

DRL + LightGBM

Deep Reinforcement Learning using initial 12 features and LightGBM forecasted realized 15 minutes

Metrics

Metric	Value	Commentary
Round-trip trades	**1,408**	Insanely high turnover
Long trades	1,045	~74% long bias
Short trades	363	~26% short
Win % (net)	**59.2%**	Costs shaved ~2.5% off
Gross PnL	**$69,100**	Signal strength
Total Costs	**$15,894**	Execution reality
Net PnL	**$53,205**	Matches equity curve exactly
Profit Factor (net)	**2.67×**	Still very strong
Avg Win (net)	$101.95	Winners still dominate
Avg Loss (net)	−$55.43	Losses contained
Max Drawdown	**$2,039**	Small relative to net PnL
Median Hold Time	1 bar	True scalping
Avg Hold Time	2.34 bars	Microstructure regime
Max Hold Time	36 bars	~36 minutes

Metric	Value	Commentary
Avg trades/day	**108**	Heavy trading

Figure 10

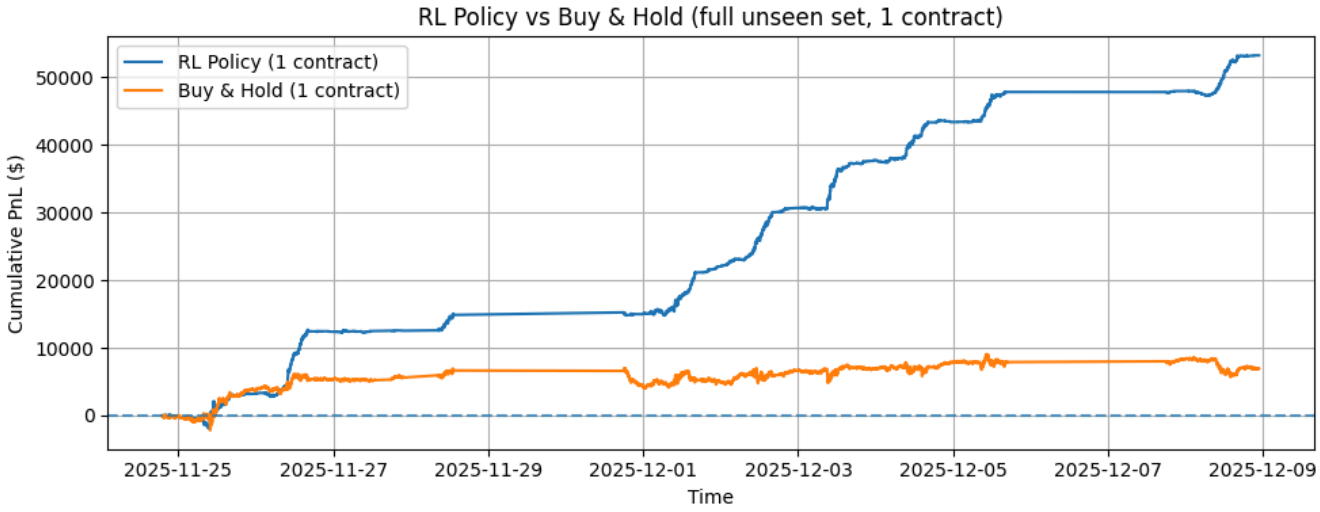

Figure 11

Figure 11 shows Equity curve of Deep Reinforcement Learning using initial 12 features and Transformer forecasted realized 15 minutes and Buy and hold

Comparison of DRL + LGB and DRL + Transformer

Metric	DRL + LGB	DRL + Transformer	Winner
Net PnL	$53,206	**$78,335**	Transformer
Profit Factor	2.67×	**4.01×**	Transformer
Win %	59.2%	**69.8%**	Transformer
Max Drawdown	$2,040	**$828**	Transformer
Trades/day	108	116	Both trade alot
Avg Hold (bars)	2.34	2.76	very similar

Figure 12

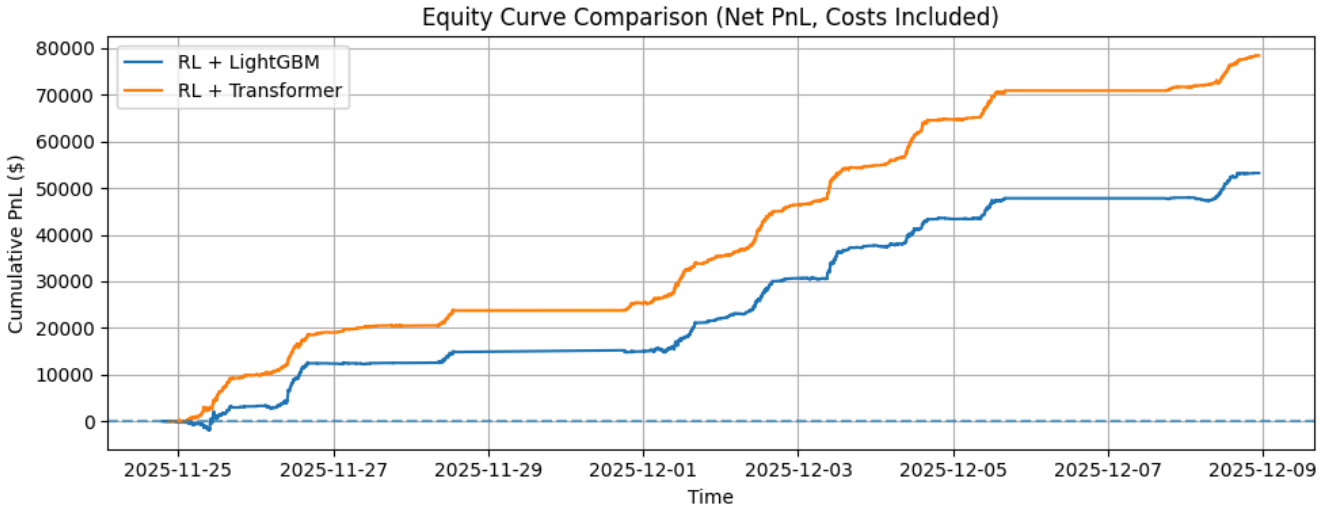

Figure 13

Figure 13 shows Equity curve RDL + LGB and DRL + Transformer

Buy & Hold (and RL) evaluation period

- Start: 2025-11-24 19:00:00
- End: 2025-12-08 22:48:00

Strategy	Net PnL
Buy & Hold	**$6,925**
RL + LGB	**$53,206**
RL + Transformer	**$78,335**

Figure 14

Discussion

Training the Volatility Models

The first step into the training was deciding which loss function to use. As stated before, MSE is symmetrical, meaning it punishes over forecast the same as under forecasting, which is very dangerous for risk management. The Training curve of of MSE does not show clear signs of heavy overfitting, both training and validation start to flat out and remain there after Epoch 6. The curves of Figure 1 tells us the training went well but when we compare the forecasted values with the realized values in Figure 2, is clear the model not only fails at forecasting some

big spikes, but it never over forecasts. If we see closely, any spike the forecasted values have, is after the realized which tells me the loss function is being too conservative.

Therefore QLIKE was used for training, in Figure 3, the training curves using QLIKE reveals that there is no heavy overfitting since training does go down and validation remains flat, but is telling me that the model may be lacking more information since validation seems stagnant. Nevertheless the models seem good enough. When we use figure 4 to see how it predicted compared to realized, now we see that the model was bold enough to overestimate in some cases and is less frequently underestimating. The reason for this is that QLIKE is asymmetrical, given the nature of the equation, it penalized very severely the under forecasting while is more forgiving with over forecasting which translates to a model that will be more willing to give big numbers while forecasting since having a big error on over forecasting is not that harmful. After showing this desired behavior, we choose QLIKE as our loss function.

The LightGBM numbers were quite interesting. The performance curve for training and validation actually have slightly better performance than QLIKE Transformer. If we compare Figure 3 and Figure 5, we see that Figure 5 has slightly less error. Also the training curves of figure 5 show there is no overfitting from the LIGHTGBM, improvement becomes very stagnant at the end and hits the early stop round 200. In figure 6, when comparing the forecasted values and the realized values, I see a similar behavior of forecasting as from the Transformer using MSE, the lightGBM Seems conservative, no over forecasting and seems it trying to follow closer the realized but always lagging.

We compared the performance in a more quantitative way from the LightGBM using QLIKE and the transformer using QLIKE. In figure 7, we have 5 different metrics, correlation simply shows how forecasted and realized align, LightGBM Wins here for the wrong reasons. It wins because is more conservative, it does not ever over forecasts which translates to higher correlation. The highest correlation of LGB is at 15 bars which is exactly 15 minutes later while the transfer is at 12 bars which tells us at least is forecasting with 3 minutes beforehand. Again, even if LGB is higher, is only because the model is not bold enough. In spike recall, transformer won slightly, this tells me that even while being a more aggressive forecaster and over forecasting, it also is able to forecast the real spikes better. At least we got precision, here LGB won again but for the reason I talked before, less aggressive forecasts, less over forecasting translate to better precision which I think is not very positive, volatility is aggressive and fast, a conservative model will react slow. Something interesting is happening here, LGB is showing some better metrics but I need to test them on the DRL to see if my sense that a more conservative model with better metrics will actually not be beneficial for trading.

Trading Models

We proceeded with the training and testing in test data on the DRL for LGB + QLIKE and Transformer + QLIKE.

DRL Transformer + QLIKE in Figure 8 showed strong numbers for Pnl with a $96,450 of profit without costs and a final $78,335 of net profit. The number alone is not that significant unless we compare it to the buy and hold of the market in the same period. It was of only $6,925 , which is a 11.31x overperformance.$100,000 was the trading capital used so it is a return of 78.3% from 24 of November of 2025 to 08 of December of 2025.

The model did trade a lot (1508) and each round trip costs $5 in total, so the total costs is an important factor to consider. The win % at 69.8% is quite realistic and makes me feel optimistic since its a stellar performance without really winning all, a desirable behavior. The model tents to long more than short (61% vs 39%) but may be due to the fact the market was mostly in an uptrend in the time period therefore the agent took advantage of it.

Watching at the AVG win of $99.14 compared to the avg loss of -$57.27 , this tell us we got a very good risk reward, the profit factor is of 4.01x, making it very robust, since it means that not only each win is much larger than each loss but also it wins more often. This two aspects are crucial to understand why the performance was this good.

Max Drawback was of only $828 which is quite low. Median bar hold is of only 1 bar, meaning the agent learned how to scalp the market mostly.

DRl LightGBM + QLIKE in Figure 10 showed strong numbers for Pnl with a $69,100 of profit without costs and a final $53,205 of net profit. When compare to the buy and hold of the market in the same period. It was of only $6,925 , which is a we have a 7.68x overperformance. $100,000 was the trading capital used so it is a return of 53.2% from 24 of November of 2025 to 08 of December of 2025.

The model did trade a lot at 1408 trades which each round trip costs $5 in total. The win % at 59.2% is realistic but getting close to the 50% of a random guess. The model tents to long the majority of the time, a lot more than short (74% vs 26%) .

Watching at the AVG win of $101.95 compared to the avg loss of -$55.43, has a good risk reward, the profit factor is of 2.67x .

Max Drawback was of only $2,039 is still low compared to the net profit. Median bar hold is of only 1 bar, meaning the agent learned how to scalp here as well.

Once we compare the two agents/architectures using the Figure 12, the transformer architecture is the clear winner, not only it out performed by $25,129 on net profit but also has a better risk profile with having a smaller max drawdown, a higher win % and a better profit factor. The transformer did trade more but was beneficial since it wins them much more often. The avg holding bars of both is very close and they both have a median of 1 bar

Potential issues and improvements

Volatility Model

The flat lining of the validation set gives me the signal that the model is not able to extract more information of the set of features it has already. We used 1 minute bars for this research, the use of more granular data like 30 seconds or even as low 1 second could create a model that reacts much faster to changes in volume and volatility. The implication of using more granular data is that more steps would be required in the Transformer and to a certain degree, if the steps are too much, a vanilla transformer will struggle with that too, so new more complex versions of the transformer may needed in that instance.

Beside more granularity to the bars, other important features could be used to improve the model, one of them is using orderbook information, which reflects the instantaneous liquidity of the market, several new features can be derived from this, very valuable features like rate of change of the resting orders, how many are being cancelled and intentions of trading. This features can help forecast volatility too and encode a lot of information the actual features just cant have.

Geo political news, Macro economic indicators and fundamental data like earnings, all are volatility generators, which are also not taken into account. This features may present a challenge to encode into features but the use of NLP with social media and news outlets can be a first step for it.

RDL and Backtest

The DRL backtest did use commissions and mimics slippage to certain degree (based on size of trade) but more realistic conditions would need to be added to make the PnL closer to reality. Some of the conditions would be latency issues, queue priority when sending an order and just getting partial fills.

The training and testing for the DRL was done in smaller spam of time due to the heavy computational time it required on a commercial GPU, so expanding this time horizon and using more professional harder is also desirable for better testing. More time spam will mean the agentic will be exposed to more regime changes and conditions, this will be necessary to see how good the agent can survive different markets and thrive on them.

Next step in the progression of the DRL agent is having it trade multiple positions and multiple targets encoded into its possible states. As for now the agent can only have a full position, exit it entirely or keep it, in a more complex decision framework, it could add to the position, could drop parts of it, this could improve the performance further by having more complex decision making to learn from.

The live deployment of this project would require complex integration with a Broker API to fetch the information directly and feed it into the project pipeline. Either run it locally or in a cloud

based environment like Google Cloud Vertex. Tight management of the profile times for feature engineering the live data, running inference and making a decision will require tight engineering, probably relying in a fast compiled language like C++ and not only python which is an interpreter and by nature, slower.

Interpretability

Both the Transformer and the DRL generate a bit of black box results, an effort to understand which features provide more value and affect the agent performance would be valuable to try to tie the result to causes. The interpretability it self can be a separate study since will require close examination.

Conclusion

From this work, an important conclusion is that volatility forecasting is not just about how accuracy looks but how it impacts the decision making down the line and how it can materialize into profit. LightGBM produced better numbers yet this did not translated to better trading performance since better accuracy was due to its conservative behavior. In contrast the transformer, traded accuracy for boldness, the asymmetric loss produced better volatility estimates for the deep reinforcement learning.

The DRL agent produce substantial higher profitability and risk profile than both the LightGBM agent and the buy and hold. This support that the use of sequence models and asymmetric loss functions (QLIKE) are a promising route for trading applications. Future work will focus on richer data sources, longer time horizons for evaluation and live deployment.

References

Generalized Autoregressive Conditional Heteroskedasticity
Bollerslev, T. (1986). *Generalized autoregressive conditional heteroskedasticity*. **Journal of Econometrics**, 31(3), 307–327.

Attention Is All You Need
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. **Advances in Neural Information Processing Systems (NeurIPS)**.

The Annotated Transformer
Rush, A. (2018). *The Annotated Transformer*. Harvard NLP.

LightGBM: A Highly Efficient Gradient Boosting Decision Tree
Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). *LightGBM: A highly efficient gradient boosting decision tree*. **Advances in Neural Information Processing Systems (NeurIPS)**.

Greedy Function Approximation: A Gradient Boosting Machine
Friedman, J. H. (2001). *Greedy function approximation: A gradient boosting machine*. **Annals of Statistics**, 29(5), 1189–1232.

Reinforcement Learning: An Introduction
Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). **MIT Press**.

Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor
Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor*. **Proceedings of the 35th International Conference on Machine Learning (ICML)**.

Soft Actor-Critic Algorithms and Applications
Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., & Levine, S. (2019). *Soft actor-critic algorithms and applications*. **arXiv preprint arXiv:1812.05905**.

Soft Actor-Critic for Discrete Action Settings
Christodoulou, P. (2019). *Soft actor-critic for discrete action settings*. **arXiv preprint arXiv:1910.07207**.